

1. Maggie is a real-estate agent and she is puzzled about why some of the properties her company manages have not been sold for more than six months. Help Maggie in providing the insights why the properties are not getting sold.

```
In [1]: # Importing Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
In [2]: # Creating a Real Estate Dataset
np.random.seed(42)
n = 200

data = pd.DataFrame({
    'price': np.random.normal(350000, 100000, n),
    'area_sqft': np.random.normal(1800, 500, n),
    'bedrooms': np.random.choice([1, 2, 3, 4, 5], n),
    'bathrooms': np.random.choice([1, 2, 3], n),
    'age_years': np.random.randint(0, 50, n),
    'distance_to_city': np.random.normal(15, 8, n),
    'parking_spaces': np.random.choice([0, 1, 2, 3], n),
    'condition_score': np.random.randint(1, 11, n),
    'neighborhood_score': np.random.randint(1, 11, n)
})

# Label: 1 = sold within 6 months, 0 = not sold
sold = []
for i in range(n):
    score = 0
    if data['price'].iloc[i] < 350000: score += 1
    if data['condition_score'].iloc[i] > 5: score += 1
    if data['neighborhood_score'].iloc[i] > 5: score += 1
    if data['distance_to_city'].iloc[i] < 15: score += 1
    if data['age_years'].iloc[i] < 20: score += 1
    sold.append(1 if score >= 3 else 0)

data['sold'] = sold
print(data.shape)
data.head()
```

(200, 10)

```
Out[2]:
```

	price	area_sqft	bedrooms	bathrooms	age_years	distance_to_city	park
0	399671.415301	1978.893680	3	3	25	7.747371	
1	336173.569883	2080.392263	1	1	40	21.755539	
2	414768.853810	2341.525622	5	1	34	14.209097	
3	502302.985641	2326.901026	4	3	24	33.383672	
4	326584.662528	1111.165316	1	1	25	24.489425	

```
In [3]: # Checking for any null values in dataset
data.isnull().sum()
```

```
Out[3]: price          0
area_sqft         0
bedrooms          0
bathrooms         0
age_years         0
distance_to_city  0
parking_spaces    0
condition_score   0
neighborhood_score 0
sold              0
dtype: int64
```

```
In [4]: # Assigned variables
feature_cols = ['price', 'area_sqft', 'bedrooms', 'bathrooms',
               'age_years', 'distance_to_city', 'parking_spaces',
               'condition_score', 'neighborhood_score']
print('Features:', feature_cols)
print('Target: sold (1 = Sold within 6 months, 0 = Not Sold)')
```

```
Features: ['price', 'area_sqft', 'bedrooms', 'bathrooms', 'age_years', 'distance_to_city', 'parking_spaces', 'condition_score', 'neighborhood_score']
Target: sold (1 = Sold within 6 months, 0 = Not Sold)
```

```
In [5]: # Creating dataframe with features only
df1 = data[feature_cols]
df1.head()
```

```
Out[5]:
```

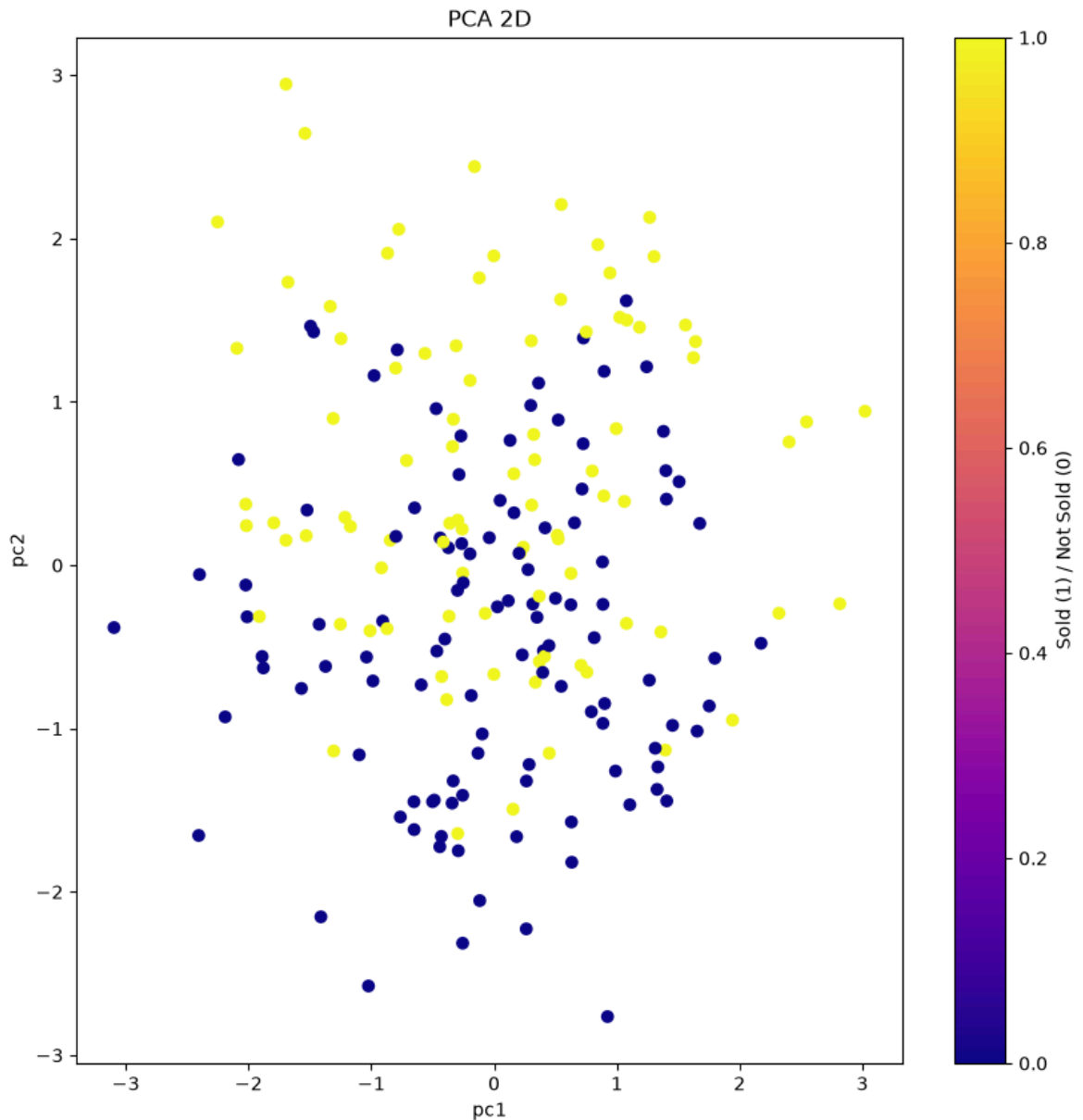
	price	area_sqft	bedrooms	bathrooms	age_years	distance_to_city	park
0	399671.415301	1978.893680	3	3	25	7.747371	
1	336173.569883	2080.392263	1	1	40	21.755539	
2	414768.853810	2341.525622	5	1	34	14.209097	
3	502302.985641	2326.901026	4	3	24	33.383672	
4	326584.662528	1111.165316	1	1	25	24.489425	

```
In [6]: # Standard Scaler and PCA
sc = StandardScaler()
sc.fit(df1)
scaled_data = sc.transform(df1)
principal = PCA(n_components=3)
principal.fit(scaled_data)
```

```
x = principal.transform(scaled_data)
print(x.shape)
```

(200, 3)

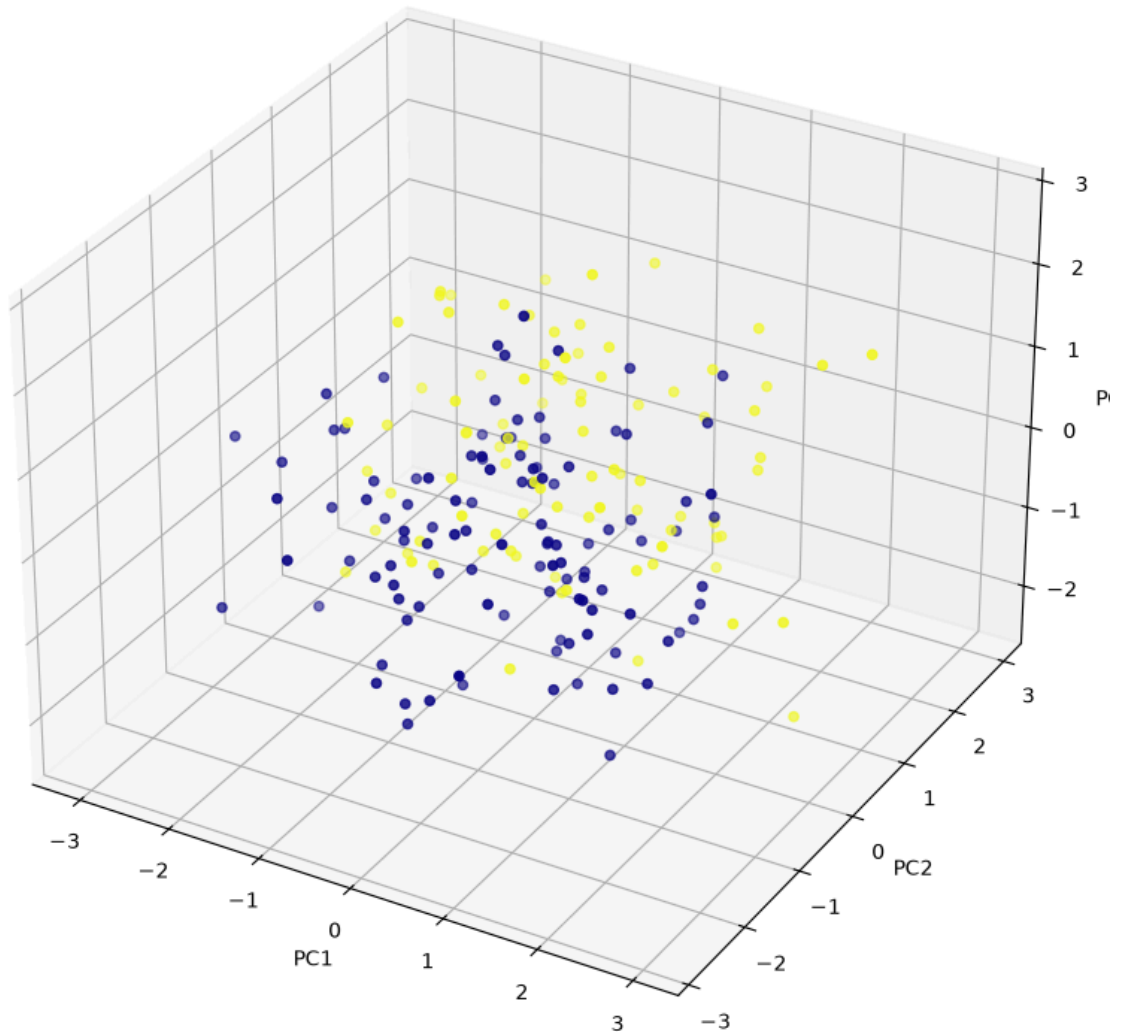
```
In [7]: # Scattering the Clusters - PCA 2D
principal.components_
plt.figure(figsize=(10, 10))
plt.title('PCA 2D')
plt.scatter(x[:, 0], x[:, 1], c=data['sold'], cmap='plasma')
plt.xlabel('pc1')
plt.ylabel('pc2')
plt.colorbar(label='Sold (1) / Not Sold (0)')
plt.show()
```



```
In [8]: # Scattering the Clusters - PCA 3D
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(10, 10))
axis = fig.add_subplot(111, projection='3d')
axis.scatter(x[:, 0], x[:, 1], x[:, 2], c=data['sold'], cmap='plasma')
axis.set_xlabel('PC1', fontsize=10)
axis.set_ylabel('PC2', fontsize=10)
axis.set_zlabel('PC3', fontsize=10)
```

```
plt.title('PCA 3D - Real Estate Properties')
plt.show()
```

PCA 3D - Real Estate Properties



```
In [9]: # Explained Variance Ratio
print('Explained Variance Ratio:', principal.explained_variance_ratio_)
print('\nTotal Variance Explained by 3 components:',
      sum(principal.explained_variance_ratio_))
```

Explained Variance Ratio: [0.13861397 0.13543535 0.11876367]

Total Variance Explained by 3 components: 0.3928129866706117

```
In [10]: # Feature Importance - Which features contribute most to each principal component
pc_df = pd.DataFrame(
    principal.components_,
    columns=feature_cols,
    index=['PC1', 'PC2', 'PC3']
)
print('Principal Component Loadings:')
pc_df
```

Principal Component Loadings:

Out[10]:	price	area_sqft	bedrooms	bathrooms	age_years	distance_to_city	parking_s
PC1	-0.263376	-0.300020	-0.100719	0.188531	0.429220	-0.324262	0.0
PC2	-0.274019	-0.452572	0.430154	-0.273700	-0.295702	-0.312581	0.4
PC3	-0.217843	-0.253348	0.321033	0.646475	0.021572	0.190629	-0.3

```
In [11]: # Insight: Properties not sold tend to have high price, old age, far from city,
# Low condition and neighborhood scores
not_sold = data[data['sold'] == 0]
sold_props = data[data['sold'] == 1]

print('=== Insights for Maggie ===')
print(f'\nTotal Properties: {len(data)}')
print(f'Sold within 6 months: {len(sold_props)}')
print(f'Not Sold: {len(not_sold)}')
print(f'\n--- Average values comparison ---')
comparison = pd.DataFrame({
    'Sold (Avg)': sold_props[feature_cols].mean(),
    'Not Sold (Avg)': not_sold[feature_cols].mean()
})
comparison
```

=== Insights for Maggie ===

Total Properties: 200
Sold within 6 months: 89
Not Sold: 111

--- Average values comparison ---

Out[11]:		Sold (Avg)	Not Sold (Avg)
	price	316737.699589	369323.652551
	area_sqft	1758.035188	1911.006189
	bedrooms	3.078652	2.855856
	bathrooms	1.988764	1.972973
	age_years	18.910112	29.333333
	distance_to_city	12.804196	18.505535
	parking_spaces	1.617978	1.495495
	condition_score	6.033708	4.837838
	neighborhood_score	6.752809	4.774775